

Software Quality and Reliability

Unit Test Project

First name: Georgios – Panagiotis

Last Name: Cheimonidis

Contents

1. CheckPhone method.....	3
2. CheckIBAN method	7
3. CheckZipCode method	12
4. CalculateSalary method	14
5. int NumofEmployees (Employee [] Empls, string Position)	21
6. bool GetBonus(ref Employee[] Empls, string Department, double IncomeGoal, double Bonus	27

1. CheckPhone method

A. Method Implementation Assumptions

- The method accepts a Phone number, checks if this number belongs to any of the countries (Greece, Cyprus, Italy, and England) and returns a true, or false value otherwise. This value is stored in PhoneCountry, which is returned from where the method was called (i.e., from the unit test).
- The number must start with either "00" and the respective country code or "+" and the respective country code (e.g., for Greece either "0030" or "+30").
- The number must not be empty (null or whitespace) or contain spaces, dashes, or parentheses.
- The supported country codes are:
 - Greece: "0030" ή "+30"
 - Cyprus: "00357" ή "+357"
 - Italy: "0039" ή "+39"
 - England: "0044" ή "+44"

B. Test Case Creation

Equivalence Classes Table

Variable	Input Condition	Equivalence Classes	
		Valid	Invalid
Phone	The number starts with "0030" or "+30"	" +306975927329", "00306973423343"	" -306973401050", "2105499201"
Phone	The number starts with "00357" or "+357"	" +35722302968", "0035799040104"	" +357 99302104", " -+357012045"
Phone	The number starts with "0039" or "+39"	" +39335031201", "003906932931"	"3910194501", " +003903241"
Phone	The number starts with "0044" or "+44"	"0044069329312", " +442665452345"	" +31101849034", "00321849034"
Phone	The number does not start with any of the above conditions or contains spaces/parentheses/dashes	-	"0030 6975927329", "0030-6975927329", "0030)6975927329", " =021991", " +111111331"

Test Cases

Test Case	Input Test Data	Expected Results	
#id	Phone	flag(bool)	PhoneCountry(string)
1	" +306975927329"	true	"Greece"
2	"00306973423343"	true	"Greece"
3	" +35722302968"	true	"Cyprus"
4	"0035799040104"	true	"Cyprus"
5	"003906932931"	true	"Italy"
6	" +39335031201"	true	"Italy"
7	"0044069329312"	true	"England"
8	" +442665452345"	true	"England"
9	" -306973401050"	false	""
10	"2105499201"	false	""
11	"35799302104"	false	""
12	" -+357012045"	false	""
13	"3910194501"	false	""
14	" +003903241"	false	""
15	" +31101849034"	false	""
16	"00321849034"	false	""
17	"0030 6975927329"	false	""
18	"0030-6975927329"	false	""
19	"0030)6975927329"	false	""
20	"=021991"	false	""
21	" +111111331"	false	""

Γ. Unit Test Development (UnitTests)

```

5  [TestClass]
6  0 references
7  public class CheckPhoneTests
8  {
9      const string greeceNumStartsfrom = "Greece starting number must be '0030' or '+30' ";
10     const string cyprusNumStartsfrom = "Cyprus starting number must be '00357' or '+357' ";
11     const string italyNumStartsfrom = "Italy starting number must be '0039' or '+39' ";
12     const string englandNumStartsfrom = "England starting number must be '0044' or '+44' ";
13
14     5 references | 0 2/5 passing
15     private void PhoneTestCases(Payroll_Lib.Payroll_Lib payroll, object[,] testcases, string expectedCountry = null)
16     {
17         bool failed = false;
18         for (int i = 0; i < testcases.GetLength(0); i++)
19         {
20             try
21             {
22                 string PhoneCountry = "";
23                 Assert.AreEqual((bool)testcases[i, 1], payroll.CheckPhone((string)testcases[i, 2], ref PhoneCountry));
24             }
25             catch (Exception ex)
26             {
27                 //the test case failed
28                 failed = true;
29                 //I report the failed test case
30                 Console.WriteLine("Failed Test Case: {0}. Reason: {1}. \n \t Hint: {2} \n \t Hint: {3}",
31                     (int)testcases[i, 0], (string)testcases[i, 2], (string)testcases[i, 3], ex.Message);
32             }
33         }
34         //In case of a failed test case it throws an exception
35         if (failed) Assert.Fail();
36     }
37 }

```

```

34 [TestMethod]
35 public void TestInvalidCharacters()
36 {
37     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
38     object[,] testcases =
39     {
40         {1, false, "0030 6975927329", "Contains space"},
41         {2, false, "0030-6975927329", "Contains dash"},
42         {3, false, "0030)6975927329", "Contains parenthesis"},
43         {4, false, "=021991", "Unrelated input"},
44         {5, false, "+3921991=", "Unrelated input"},
45         {6, false, "+111111331", "Unrelated input"},
46     };
47     PhoneTestCases(payroll, testcases);
48 }
49 [TestMethod]
50 public void TestGreekPhoneCode()
51 {
52     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
53     object[,] testcases =
54     {
55         {1, true, "+306975927329", greeceNumStartsfrom},
56         {2, true, "00306973423343", greeceNumStartsfrom},
57         {3, false, "-306973481050", greeceNumStartsfrom},
58         {4, false, "2105499281", greeceNumStartsfrom},
59     };
60     PhoneTestCases(payroll, testcases, "Greece");
61 }

```

```

62 [TestMethod]
63 public void TestCyprusPhoneCode()
64 {
65     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
66     object[,] testcases =
67     {
68         {1, true, "+35722302968", cyprusNumStartsfrom},
69         {2, true, "0035799040104", cyprusNumStartsfrom},
70         {3, false, "+357 99302104", cyprusNumStartsfrom},
71         {4, false, "+357012045", cyprusNumStartsfrom},
72     };
73     PhoneTestCases(payroll, testcases, "Cyprus");
74 }
75 [TestMethod]
76 public void TestItalyPhoneCode()
77 {
78     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
79     object[,] testcases =
80     {
81         {1, true, "+39335031201", italyNumStartsfrom},
82         {2, true, "003906932931", italyNumStartsfrom},
83         {3, false, "3910194501", italyNumStartsfrom},
84         {4, false, "+003903241", italyNumStartsfrom},
85     };
86     PhoneTestCases(payroll, testcases, "Italy");
87 }

```

```

88 [TestMethod]
89 public void TestEnglandPhoneCode()
90 {
91     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
92     object[,] testcases =
93     {
94         {1, true, "0044069329312", englandNumStartsfrom},
95         {2, true, "0035799040104", englandNumStartsfrom},
96         {3, false, "35799302104", englandNumStartsfrom},
97         {4, false, "-+357012045", englandNumStartsfrom},
98     };
99     PhoneTestCases(payroll, testcases, "England");
100 }
101 }

```

A. Test Reports

Test Explorer

Test run finished: 5 Tests (2 Passed, 3 Failed, 0 Skipped) run in 419 ms

Test	Duration	Traits	Error Message
Payroll_Lib_UnitTests (5)	94 ms		
Payroll_Lib_UnitTests (5)	94 ms		
CheckPhoneTests (5)	94 ms		
TestCyprusPhoneCode	< 1 ms		Assert.Fail failed.
TestEnglandPhoneCode	1 ms		Assert.Fail failed.
TestGreekPhoneCode	< 1 ms		
TestInvalidCharacters	93 ms		Assert.Fail failed.
TestItalyPhoneCode	< 1 ms		

Group Summary

Payroll_Lib_UnitTests

Tests in group: 5

Total Duration: 94 ms

Outcomes

2 Passed

3 Failed

Test Detail Summary

TestCyprusPhoneCode

Source: CheckPhoneTests.cs line 63

Duration: < 1 ms

Message: Assert.Fail failed.

Stack Trace:

CheckPhoneTests.PhoneTestCases(Payroll_Lib payroll, Object[,] testcases, String expectedCountry) line 32

CheckPhoneTests.TestCyprusPhoneCode() line 73

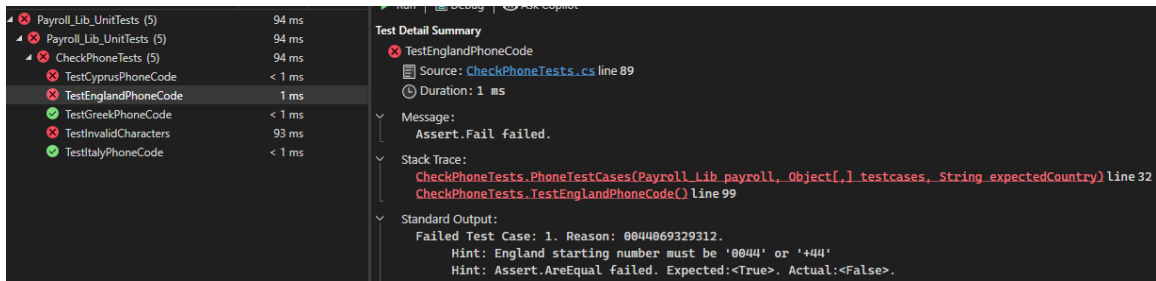
Standard Output:

Failed Test Case: 1. Reason: +35722302968.

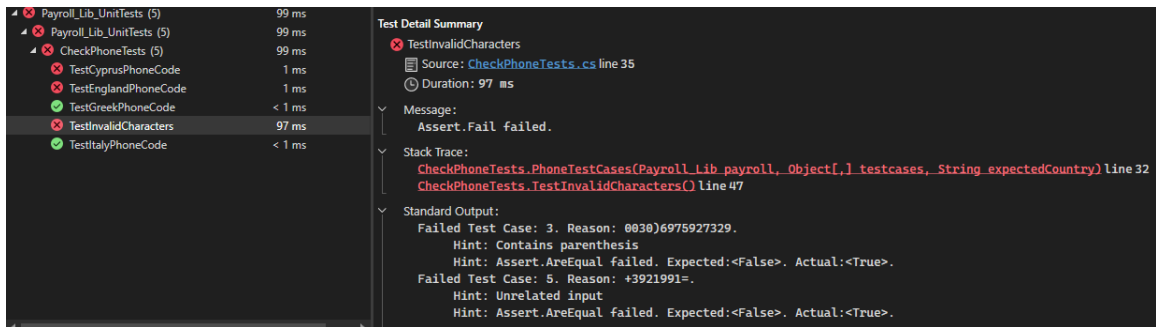
Hint: Cyprus starting number must be '00357' or '+357'

Hint: Assert.AreEqual failed. Expected:<True>. Actual:<False>.

As shown in TestMethod = TestCyprusPhoneCode, it throws an error for checking the number +35722302968. Normally it should not throw an error because it starts correctly with +357 (1 of the 2 cases with which the Cyprus phone number must start). This means that the programmer has set it to check the wrong country code, which is a logical error. Therefore, where Actual should be True, it is False.



As shown in TestMethod = TestEnglandPhoneCode, it throws a similar error to the previous method for checking the number 0044069329312. Normally it should not throw an error because it correctly starts with 0044 (1 of the 2 cases with which the England phone number must start). This means that the programmer has made the same logical error.



As shown in TestMethod = TestInvalidCharacters, 2 errors have occurred. The 1st is the presence of the right parenthesis: 0030)6975927329, where normally a restriction should have been added by the programmer so it cannot be accepted. The 2nd is the presence of "Unrelated input" in the number. Namely, a character (outside the trio of spaces/parentheses/dashes) has been entered that is not acceptable. This character is '='. (Comment: normally it is not stated in the method's specifications that it must be checked, but it was mentioned because normally it is also an error case).

2. CheckIBAN method

A. Method Implementation Assumptions

- The CheckIBAN method accepts a string representing the IBAN number as input.
- It checks if the IBAN number is valid based on the IBAN bank account number format validation, returning true if it is, and false otherwise.
- The country to which the account belongs will be returned via the by ref IBANCountry parameter.
- Only IBANs from the countries Greece, Cyprus, Italy, and England are allowed.
- For IBAN validation, the initial country prefixes (Greece, Italy, Cyprus, England) are GR, IT, CY, GB respectively. Furthermore, for correctness, the IBAN bank account validation check indicated [here](#) must be followed.

B. Test Case Creation

Πίνακας Ισοδύναμων Κλάσεων

Variable	Input Condition	Equivalence Classes	
		Valid	Invalid
Greece/Italy IBAN	Greece: starts with "GR" and has a length of 27. Italy: starts with "IT" and has a length of 27	<u>Greece</u> : "GR1601101250000000012300695" <u>Italy</u> : "IT60X0542811101000000123456"	<u>Greece</u> : "GR160110125000000001230069", "GR16011012500000000123006951", "GR1601101250000000012300696" <u>Italy</u> : "IT60Y054281110100000012345", "IT60Y05428111010000001234561", "IT60Y0542811101000000123457"
Cyprus IBAN	starts with "CY" and has a length of 28	<u>Cyprus</u> : "CY17002001280000001200527600"	"CY1700200128000000120052760", "CY170020012800000012005276001", "CY17002001280000001200527601"
England IBAN	ξεκινά με "GB" και έχει μήκος 22	<u>England</u> : "GB94BARC10201530093459"	"GB94BARC1020153009345", "GB94BARC102015300934591", "GB94BARC10201530093460"
IBAN	starts with an unaccepted country prefix and its length is out of bounds or empty or another country	-	" " or null, "FR7630006000011234567890189"

Test Cases

<i>Test Cases</i>	<i>Input Test Data</i>	<i>Expected Results</i>	
<i>#id</i>	<i>IBAN</i>	<i>flag(bool)</i>	<i>IBANCountry(string)</i>
1	"GR1601101250000000012300695"	true	"Greece"
2	"GR160110125000000001230069" (length:26)	false	""
3	"GR16011012500000000123006951" (length: 28)	false	""
4	"GR1601101250000000012300696" (division 97<>1)	false	""
5	"IT60X0542811101000000123456"	true	"ITALY"
6	"IT60Y054281110100000012345" (length:26)	false	""
7	"IT60Y05428111010000001234561" (length:28)	false	""
8	"IT60Y0542811101000000123457" (division 97<>1)	false	""
9	"CY17002001280000001200527600"	true	"CYPRUS"
10	"CY1700200128000000120052760" (length:27)	false	""
11	"CY170020012800000012005276001" (length:29)	false	""
12	CY17002001280000001200527601 (division 97<>1)	false	""
13	"GB94BARC10201530093459"	true	"ENGLAND"
14	"GB94BARC1020153009345" (length:21)	false	""
15	"GB94BARC102015300934591" (length:23)	false	""
16	"GB94BARC10201530093460" (division 97<>1)	false	""
17	" " or null	false	""
18	"FR7630006000011234567890189"	false	""

F. Unit Test Development (UnitTests)

```
102 [TestClass]
103 0 references
104 public class CheckIBANTests
105 {
106     5 references | 2/5 passing
107     private void IBANTestCases(Payroll_Lib.Payroll_Lib payroll, object[,] testcases, string expectedCountry)
108     {
109         bool failed = false;
110         for (int i = 0; i < testcases.GetLength(0); i++)
111         {
112             try
113             {
114                 string IBANCountry = "";
115                 Assert.AreEqual((bool)testcases[i, 1], payroll.CheckIBAN((string)testcases[i, 2], ref IBANCountry));
116                 if ((bool)testcases[i, 1] == true)
117                 {
118                     Assert.AreEqual(expectedCountry, IBANCountry);
119                 }
120             }
121             catch (Exception ex)
122             {
123                 //the test case failed
124                 failed = true;
125                 //I report the failed test case
126                 Console.WriteLine("Failed Test Case: {0}. Reason: {1}. \n \t Hint: {2} \n \t Hint: {3}",
127                     (int)testcases[i, 0], (string)testcases[i, 2], (string)testcases[i, 3], ex.Message);
128             }
129         }
130         //In case of a failed test case it throws an exception
131         if (failed) Assert.Fail();
132     }
133 }
134
135 const string greeceCountryCode = "Greece IBAN must start with 'GR'";
136 const string greeceLength = "IBAN must be 27 characters long. \n \t Not higher or less. The IBAN is not valid.";
137 const string italyCountryCode = "Italy IBAN must start with 'IT'";
138 const string italyLength = "IBAN must be 27 characters long. \n \t Not higher or less. The IBAN is not valid.";
139 const string cyprusCountryCode = "Cyprus IBAN must start with 'CY'";
140 const string cyprusLength = "IBAN must be 28 characters long. \n \t Not higher or less. The IBAN is not valid.";
141 const string englandCountryCode = "England IBAN must start with 'GB'";
142 const string englandLength = "IBAN must be 22 characters long. Not higher or less. \n \t The IBAN is not valid.";
143 const string UnacceptableDivResult = "Unacceptable division result. The remainder isn't 1. \n \t The IBAN is not valid.";
144 [TestMethod]
145 0 references
146 public void TestGreeceIBAN()
147 {
148     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
149     object[,] testcases =
150     {
151         {1, true, "GR1601101250000000012300695", greeceLength + " and " + greeceCountryCode},
152         {2, false, "GR160110125000000001230069", greeceLength},
153         {3, false, "GR16011012500000000123006951", greeceLength},
154         {4, false, "GR1601101250000000012300696", UnacceptableDivResult},
155     };
156     IBANTestCases(payroll, testcases, "Greece");
157 }
158
159 [TestMethod]
160 0 references
161 public void TestItalyIBAN()
162 {
163     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
164     object[,] testcases =
165     {
166         {1, true, "IT60X0542811101000000123456", italyCountryCode + " and " + italyLength},
167         {2, false, "IT60X054281110100000012345", italyLength},
168         {3, false, "IT60X05428111010000001234561", italyLength},
169         {4, false, "IT60X0542811101000000123457", UnacceptableDivResult},
170     };
171     IBANTestCases(payroll, testcases, "Italy");
172 }
```

```

168 public void TestCyprusIBAN()
169 {
170     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
171     object[,] testcases =
172     {
173         {1, true, "CY17002001280000001200527600", cyprusCountryCode+ " and "+cyprusLength},
174         {2, false, "CY1700200128000000120052760", cyprusLength},
175         {3, false, "CY170020012800000012005276001", cyprusLength},
176         {4, false, "CY17002001280000001200527601", UnacceptableDivResult},
177     };
178     IBANTestCases(payroll, testcases, "Cyprus");
179 }
180 [TestMethod]
181 public void TestEnglandIBAN()
182 {
183     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
184     object[,] testcases =
185     {
186         {1, true, "GB94BARC10201530093459", englandCountryCode+ " and "+englandLength},
187         {2, false, "GB94BARC1020153009345", englandLength},
188         {3, false, "GB94BARC102015300934591", englandLength},
189         {4, false, "GB94BARC10201530093460", UnacceptableDivResult},
190     };
191     IBANTestCases(payroll, testcases, "England");
192 }
193 [TestMethod]
194 public void TestUnsupportedOrEmptyIBAN()
195 {
196     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
197     object[,] testcases =
198     {
199         {1, false, " ", "Empty IBAN should not be valid."},
200         {2, false, "FR7630006000011234567890189", "France IBAN not supported in this implementation."},
201     };
202     IBANTestCases(payroll, testcases, "");
203 }

```

Δ. Test Reports

Test run finished: 5 Tests (1 Passed, 4 Failed, 0 Skipped) run in 445 ms

Test	Duration	Trace
Payroll_Lib_UnitTests (11)	192 ms	
Payroll_Lib_UnitTests (11)	192 ms	
CheckIBANTests (5)	96 ms	
TestCyprusIBAN	1 ms	
TestEnglandIBAN	88 ms	
TestGreeceIBAN	7 ms	
TestItalyIBAN	< 1 ms	
TestUnsupportedOrEmptyIBAN	< 1 ms	

Group Summary

CheckIBANTests

Tests in group: 5

⌚ Total Duration: 96 ms

Outcomes

✅ 1 Passed

❌ 4 Failed

Test Detail Summary

❌ TestCyprusIBAN

Source: CheckPhoneTests.cs line 168

⌚ Duration: 1 ms

Message:

Assert.Fail failed.

Stack Trace:

CheckIBANTests.IBANTestCases(Payroll_Lib payroll, Object[,] testcases, String expectedCountry) line 129

CheckIBANTests.TestCyprusIBAN() line 178

Standard Output:

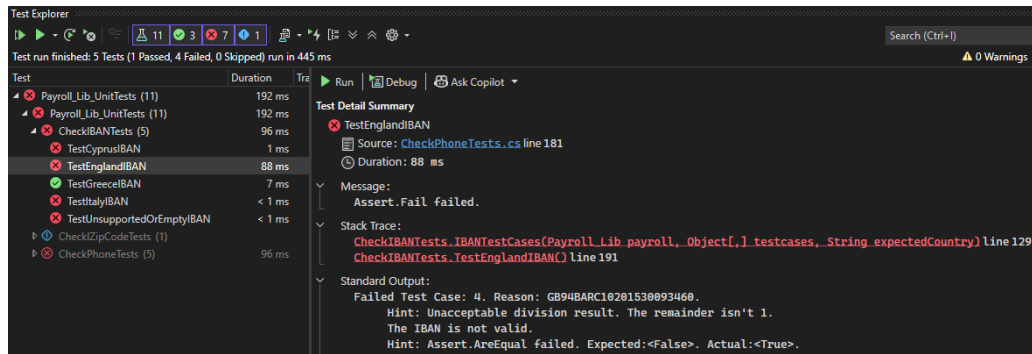
Failed Test Case: 1. Reason: CY17002001280000001200527600.

Hint: Cyprus IBAN must start with 'CY' and IBAN must be 28 characters long.

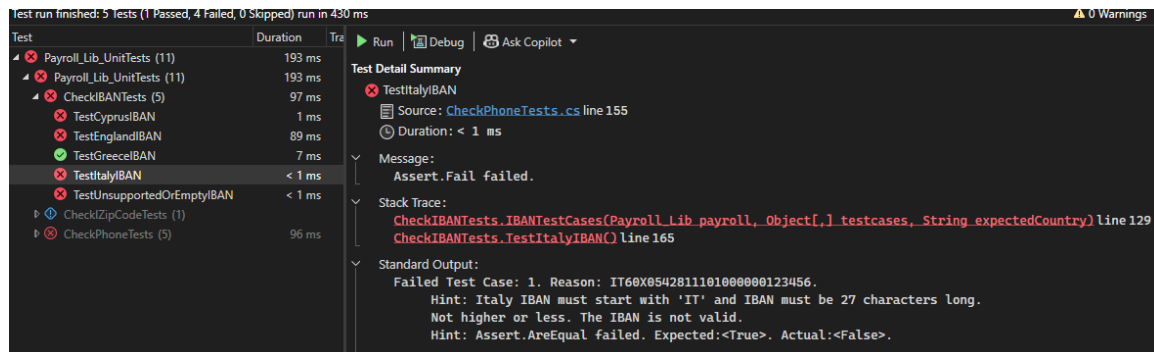
Not higher or less. The IBAN is not valid.

Hint: Assert.AreEqual failed. Expected:<True>. Actual:<False>.

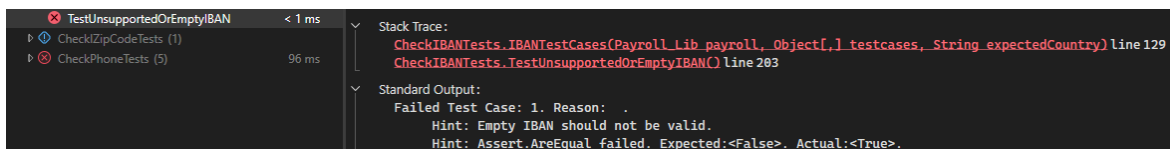
As shown in TestMethod = TestCyprusIBAN, it throws an error for checking the IBAN: CY17002001280000001200527600. Normally it should not throw an error because it correctly starts with CY (1 of the 2 cases for testing the IBAN). This means that the programmer has placed the wrong initial prefix check for the country, which is a logical error. Therefore, where Actual should be True, it is False.



As shown in TestMethod = TestEnglandIBAN, it throws an error for checking the IBAN: GB94BARC10201530093460. Normally it should not throw an error, because the IBAN bank account number validation check is correct and it should output a remainder of 1 in the division based on validity (1 of the 2 cases for testing the IBAN). This means that the programmer has made a mistake in the validation check or does not include it for the respective country (which is the case here), representing a logical error. Therefore, where Actual should be True, it is False.



As shown in TestMethod = TestItalyIBAN, it throws an error for checking the IBAN: IT60X0542811101000000123456. Normally it should not throw an error, because it is indeed 27 characters. This means that the programmer has made an error in checking the character length of Italy's IBAN, which is a logical error. Therefore, where Actual should be True, it is False.



As shown in TestMethod = TestItalyIBAN, it throws an error for checking the IBAN: IT60X0542811101000000123456. Normally it should not throw an error, because it is indeed empty. This means that the programmer has made a mistake in the empty check. (Note: translated verbatim from the provided source document text).

3. CheckZipCode method

A. Method Implementation Assumptions

- This function accepts an integer ZipCode, which refers to an Italian address, and returns whether it is valid.
- They are always 5-digit numbers (e.g., 00184 for Rome) and their value range is from 00010 to 98168.

B. Test Case Creation

Equivalence Classes Table

Variable	Input Condition	Equivalence Classes	
		Valid	Valid
ZipCode	It is a 5-digit number and is within the valid Italian ZipCode range (00010 to 98168)	10000	00011
ZipCode	It is a 5-digit number but is not within the Italian ZipCode range (00010 to 98168)	-	00009,-50000,98169
ZipCode	It is not a 5-digit number	-	93291039

Test Cases

Test Cases	Input Test Data	Expected Results
#id	ZipCode(int)	flag(bool)
1	10000	true
2	00011 (it is an int value and gets converted to 11, so it cannot be checked)	false
3	00009 (minimum boundary value)	false
4	-50000	false
5	98169 (maximum boundary value)	false
6	93291039(non 5-digit number)	false

Γ. Unit Test Development (UnitTests)

```
207 public class CheckZipCodeTests
208 {
209     [TestMethod]
210     public void TestItalianZipCodes()
211     {
212         Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
213
214         object[,] testcases =
215         {
216             {1, true, 10000, "The ZipCode must be a 5-digit number"},
217             {2, true, 00011, "The ZipCode must be a 5-digit number"},
218             {3, false, 00009, "Below minimum. ZipCode must > 10 and < 98168."},
219             {4, false, -50000, " It is a 5-digit number but it's negative zip therefor invalid."},
220             {5, false, 98169, "Above maximum. ZipCode must > 10 and < 98168."},
221             {6, false, 93291039, "The ZipCode must be a 5-digit number"},
222         };
223
224         bool failed = false;
225         for (int i = 0; i < testcases.GetLength(0); i++)
226         {
227             try
228             {
229                 Assert.AreEqual((bool)testcases[i, 1], payroll.CheckZipCode((int)testcases[i, 2]));
230             }
231             catch (Exception ex)
232             {
233                 // the test case failed
234                 failed = true;
235                 // I report the failed test case
236                 Console.WriteLine("Failed Test Case: {0}. Reason: {1}. \n \t Hint: {2} \n \t Hint: {3}",
237                     (int)testcases[i, 0], (int)testcases[i, 2], (string)testcases[i, 3], ex.Message);
238             }
239         }
240         // In case of a failed test case it throws an exception
241         if (failed) Assert.Fail();
242     }
243 }
```

Δ. Test Reports

The screenshot shows the Test Explorer window with a test run summary at the top: "Test run finished: 1 Tests (0 Passed, 1 Failed, 0 Skipped) run in 437 ms". Below this is a table of test results:

Test	Duration
Payroll_Lib_UnitTests (11)	298 ms
Payroll_Lib_UnitTests (11)	298 ms
CheckIBANTests (5)	103 ms
CheckZipCodeTests (1)	99 ms
TestItalianZipCodes	99 ms
CheckPhoneTests (5)	96 ms

On the right, the "Test Detail Summary" for the failed test "TestItalianZipCodes" is shown. It indicates the source as "CheckPhoneTests.cs line 210" and the duration as "99 ms". The message states "Assert.Fail failed." The stack trace points to "CheckZipCodeTests.TestItalianZipCodes() line 241". The standard output shows the failure details: "Failed Test Case: 2. Reason: 11. Hint: The ZipCode must be a 5-digit number Hint: Assert.AreEqual failed. Expected:<True>. Actual:<False>."

As shown in TestMethod = TestItalianZipCodes, it throws an error for checking the ZipCode: 11 (testing 00011). The problem here lies within the exercise specifications, specifically that the ZipCode argument of CheckZipCode is of type int, meaning a number with leading zeros before the number 11 cannot be passed because they are lost. Normally, it should be of type string in order to pass the value 00011 from the test method to the dll library.

4. CalculateSalary method

A. Method Implementation Assumptions

- The method accepts the details of an employee (Employee struct) as input, and calculates and returns (via ref parameters) their gross, net, and annual net earnings, as well as their final income tax and insurance contributions.
- The Employee struct containing the employee's details is as follows:
 - ❖ FirstName
 - ❖ Surname
 - ❖ Children (number of children)
 - ❖ Department (company department: Public Works, Banking Projects, Networks)
 - ❖ Position (job position: Junior Developer, Mid-level Developer, Senior Developer, IT Manager)
 - ❖ WorkExperience (years of experience, 0–38)
 - ❖ Income (revenue for bonus calculation - used in the next method)
 - ❖ Bonus (bonus amount - calculated in the next method)
- The basic monthly salary is chosen by the programmer and must be within the bounds of the following job positions:
 - Junior Developer: 1.000 – 1.400 euros monthly
 - Mid-level Developer: 1.500 – 2.000 euros monthly
 - Senior Developer: 2.000 – 2.800 euros monthly
 - IT Manager: 3.500 – 5.000 euros monthly
- The calculations for gross/net earnings, income tax (progressive taxation, deductions based on children), and insurance contributions (13.4% on the gross salary for 14 salaries) are based on the instructions described [here](#).

B. Test Case Creation

Equivalence Classes Table

Variable	Input Condition	Equivalence Classes	
		Valid	Invalid
Number of children(Children)	Type: int in a valid range of values. If the children are more than 4, they fall under the same category as 4 children (regarding the tax deduction)	Positive values from (0 and above)	-1
Company department (Department)	Type: string and takes specific positions	"Junior Developer", "Mid-level Developer", "Senior Developer", "IT Manager"	Null or other
Job position (Position)	Type: string and takes specific positions	"Public Works", "Banking Projects", "Networks"	Null or other
Years of experience (WorkExperience)	Type: int in a valid range of values	0 to 38	-1, 39

Test Cases

Test Cases	Input Test Data				Expected Results					
#id	Children	Department	Position	Work Experience	flag(bool)	AnnualGrossSalary	NetAnnualIncome	NetMonthIncome	Tax	Insurance
1	3	Banking Projects	Junior Developer	3	true	16.786 €	13657,47 €	975,53 €	800,82 €	2327,71 €
2	0	Networks	Senior Developer	7	true	39.200€	26866,05 €	1.918,99 €	6898,09 €	5435,86 €
3	7	Banking Projects	IT Manager	-1	false	0	0	0	0	0
4	2	Public Works	IT Manager	40	false	0	0	0	0	0
5	4	Office Clerk	Mid-level Developer	15	false	0	0	0	0	0
6	-1	Networks	Junior Developer	0	false	0	0	0	0	0
7	0	Networks	Manager	0	false	0	0	0	0	0

Γ. Unit Test Development (UnitTests)

```
245 [TestClass]
246 0 references
247 public class CalculateSalaryTests
248 {
249     private const double Delta = 0.05; // ανοχή για στρογγυλοποιήσεις (0.01€)
250
251     7 references | 1/7 passing
252     private void CalculateSalaryTestsCases(Payroll_Lib.Payroll_Lib payroll, object[,] testcases)
253     {
254         bool failed = false;
255         for (int i = 0; i < testcases.GetLength(0); i++)
256         {
257             int caseId = (int)testcases[i, 0];
258             try
259             {
260                 bool expectedFlag = (bool)testcases[i, 1];
261                 Employee EmpIX = (Employee)testcases[i, 2];
262                 double expectedAnnualGross = (double)testcases[i, 3];
263                 double expectedNetAnnual = (double)testcases[i, 4];
264                 double expectedNetMonth = (double)testcases[i, 5];
265                 double expectedTax = (double)testcases[i, 6];
266                 double expectedInsurance = (double)testcases[i, 7];
267
268                 double AnnualGrossSalary = 0;
269                 double NetAnnualIncome = 0;
270                 double NetMonthIncome = 0;
271                 double Tax = 0;
272                 double Insurance = 0;
273
274                 bool actualFlag = payroll.CalculateSalary(
275                     EmpIX,
276                     ref AnnualGrossSalary,
277                     ref NetAnnualIncome,
278                     ref NetMonthIncome,
279                     ref Tax,
280                     ref Insurance);
281
282                 string errorMsg = "";
283
284             }
```

```
285 // This is the case where I expect success and I have success
286 // I validate numeric results to ensure they match the expected ones.
287 // If there is any mismatch, detailed error messages are collected.
288 if (expectedFlag && actualFlag)
289 {
290     if (AnnualGrossSalary != expectedAnnualGross)
291         errorMsg += $" - AnnualGrossSalary: expected: {expectedAnnualGross}, actual: {AnnualGrossSalary}\n";
292     if (NetAnnualIncome != expectedNetAnnual)
293         errorMsg += $" - NetAnnualIncome: expected: {expectedNetAnnual}, actual: {NetAnnualIncome}\n";
294     if (NetMonthIncome != expectedNetMonth)
295         errorMsg += $" - NetMonthIncome: expected: {expectedNetMonth}, actual: {NetMonthIncome}\n";
296     if (Tax != expectedTax)
297         errorMsg += $" - Tax: expected: {expectedTax}, actual: {Tax}\n";
298     if (Insurance != expectedInsurance)
299         errorMsg += $" - Insurance: expected: {expectedInsurance}, actual: {Insurance}\n";
300
301     if ((EmpIX.Position == "Junior Developer" || EmpIX.Position == "Senior Developer" || EmpIX.Position == "Mid-level Developer" ||
302         EmpIX.Position == "IT Manager") && (AnnualGrossSalary != expectedAnnualGross || NetAnnualIncome != expectedNetAnnual ||
303         NetMonthIncome != expectedNetMonth))
304     {
305         if (EmpIX.Position == "Junior Developer")
306             errorMsg += " - Calculations for position 'Junior Developer' do not match the expected specification values.\n" +
307                 "    Verify if logical errors exist.";
308         else if (EmpIX.Position == "Mid-level Developer")
309             errorMsg += " - Calculations for position 'Mid-level Developer' do not match the expected specification values.\n" +
310                 "    Verify if logical errors exist.";
311         else if (EmpIX.Position == "Senior Developer")
312             errorMsg += " - Calculations for position 'Senior Developer' do not match the expected specification values.\n" +
313                 "    Verify if logical errors exist.";
314         else
315             errorMsg += " - Calculations for position 'IT Manager' do not match the expected specification values.\n" +
316                 "    Verify if logical errors exist.";
317     }
318 }
```

```

320 // This is the case where I expect failure and I have failure
321 // I verify that the employee data is outside valid ranges and that all amounts of the results remain zero.
322 // If not, descriptive error messages explain why the case should have failed.
323 if (actualFlag != expectedFlag)
324 {
325     if (EmpIX.Department != "Τραπεζικών Έργων" && EmpIX.Department != "Διεύθυνση" && EmpIX.Department != "Ανθρώπινου Έργων")
326         errorMsg += $" - The Department \"{EmpIX.Department}\" is not among the allowed ones:\n" +
327             $" 'Ανθρώπινου Έργων'\n" +
328             $" 'Τραπεζικών Έργων'\n" +
329             $" 'Διεύθυνση'\n";
330     if (EmpIX.Position != "Junior Developer" && EmpIX.Position != "Senior Developer" && EmpIX.Position != "Mid-level Developer" &&
331         EmpIX.Position != "IT Manager")
332         errorMsg += $" - The Position \"{EmpIX.Position}\" is not among the allowed ones:\n" +
333             $" 'Junior Developer'\n" +
334             $" 'Mid-level Developer'\n" +
335             $" 'Senior Developer'\n" +
336             $" 'IT Manager'\n";
337     if (EmpIX.workExperience < 0 || EmpIX.workExperience > 38)
338         errorMsg += $" - Work experience ({EmpIX.workExperience}) is outside the acceptable range (0-38).\n";
339     if (EmpIX.Children < 0)
340         errorMsg += $" - The number of children ({EmpIX.Children}) cannot be negative.\n";
341
342     if (AnnualGrossSalary != 0 || NetAnnualIncome != 0 || NetMonthIncome != 0 || Tax != 0 || Insurance != 0)
343         errorMsg += $" - Therefore, all amounts should have been 0. Instead they are: " +
344             $"AnnualGrossSalary={AnnualGrossSalary}, " +
345             $"NetAnnualIncome={NetAnnualIncome}, " +
346             $"NetMonthIncome={NetMonthIncome}, " +
347             $"Tax={Tax}, " +
348             $"Insurance={Insurance}\n";
349
350     //Neutral informational notes for invalid inputs
351     if (EmpIX.workExperience < 0)
352         errorMsg += " - Verify that negative Work Experience is rejected.\n";
353     if (EmpIX.workExperience > 38)
354         errorMsg += " - Verify that Work Experience > 38 is rejected.\n";
355     if (EmpIX.Children < 0)
356         errorMsg += " - Verify that negative children count is rejected.\n";
357     if (EmpIX.Position != "Junior Developer" && EmpIX.Position != "Senior Developer" && EmpIX.Position
358         != "Mid-level Developer" && EmpIX.Position != "IT Manager")
359         errorMsg += " - Verify that invalid Position is rejected.\n";
360     if (EmpIX.Department != "Τραπεζικών Έργων" && EmpIX.Department != "Διεύθυνση" &&
361         EmpIX.Department != "Ανθρώπινου Έργων")
362         errorMsg += " - Verify that invalid Department is rejected.\n";
363 }
364

```

```

366     if (!string.IsNullOrEmpty(errorMsg))
367         Assert.IsTrue(false, errorMsg);
368 }
369 catch (Exception ex)
370 {
371     failed = true;
372     Console.WriteLine(" Failed Test Case: {0}. \n Reason: {1}", caseId, ex.Message);
373 }
374 // In case of a failed test case it throws an exception
375 if (failed) Assert.Fail();
376 }
377 }
378

```

```

380 [TestMethod]
381 public void TestCase_id_1()
382 {
383     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
384     object[,] testcases =
385     {
386         {
387             1, // caseId
388             true, // expectedFlag for success
389             new Employee("Giorgos", "Chelmonidis", 3, "Τραπεζικών Έργων", "Junior Developer", 3),
390             16786.0, // expected AnnualGrossSalary
391             13687.47, // expected NetAnnualIncome
392             975.53, // expected NetMonthIncome
393             800.02, // expected Tax
394             2327.71 // expected Insurance
395         }
396     };
397     CalculateSalaryTestsCases(payroll, testcases);
398 }
399
400 [TestMethod]
401 public void TestCase_id_2()
402 {
403     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
404     object[,] testcases =
405     {
406         {
407             2, // caseId
408             true, // expectedFlag for success
409             new Employee("Materina", "Giorgatzi", 0, "Διεύθυνση", "Senior Developer", 7),
410             39200.0, // expected AnnualGrossSalary
411             26866.05, // expected NetAnnualIncome
412             1918.99, // expected NetMonthIncome
413             6898.09, // expected Tax
414             5435.86, // expected Insurance
415         }
416     };
417     CalculateSalaryTestsCases(payroll, testcases);
418 }
419
420

```

```

422 [TestMethod]
423 public void TestCase_id_3()
424 {
425     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
426     object[,] testcases =
427     {
428         {
429             3, // caseId
430             false, // expectedFlag for success
431             new Employee("Nikos", "Kalofwnos", 7, "Τραπεζικών Έργων", "IT Manager", -1),
432             0.0, // expected AnnualGrossSalary
433             0.0, // expected NetAnnualIncome
434             0.0, // expected NetMonthIncome
435             0.0, // expected Tax
436             0.0 // expected Insurance
437         }
438     };
439     CalculateSalaryTestsCases(payroll, testcases);
440 }
441
442 [TestMethod]
443 public void TestCase_id_4()
444 {
445     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
446     object[,] testcases =
447     {
448         {
449             4, // caseId
450             false, // expectedFlag for success
451             new Employee("Christos", "Skiadaresis", 2, "Ανθρώπινου Έργων", "IT Manager", 39),
452             0.0, // expected AnnualGrossSalary
453             0.0, // expected NetAnnualIncome
454             0.0, // expected NetMonthIncome
455             0.0, // expected Tax
456             0.0 // expected Insurance
457         }
458     };
459     CalculateSalaryTestsCases(payroll, testcases);
460 }
461
462

```

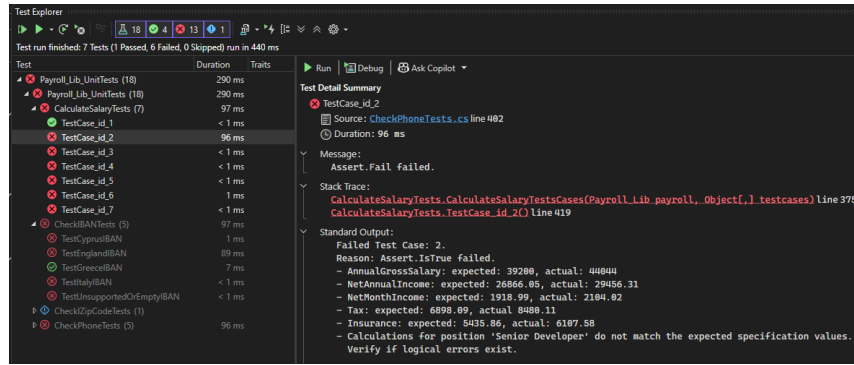
```

464 [TestMethod]
465 // 0 references
466 public void TestCase_id_5()
467 {
468     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
469
470     object[,] testcases =
471     {
472         {
473             5, // caseId
474             false, // expectedFlag for success
475             new Employee("Chrysa", "Chardaloupa", 4, "Υπόλληλος γραφείου", "Mid-level Developer", 15),
476             0.0, // expected AnnualGrossSalary
477             0.0, // expected NetAnnualIncome
478             0.0, // expected NetMonthIncome
479             0.0, // expected Tax
480             0.0, // expected Insurance
481         }
482     };
483     CalculateSalaryTestsCases(payroll, testcases);
484 }
485
486 [TestMethod]
487 // 0 references
488 public void TestCase_id_6()
489 {
490     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
491
492     // Collection of test cases for TestCase_id_2,3,4,5 with expected value 0 for all salary components
493     object[,] testcases =
494     {
495         // Case 6: Mid-level Developer, Children=4, WorkExp=15, Department=Υπόλληλος γραφείου (άκυρο)
496         {
497             6, // caseId
498             false, // expectedFlag for success
499             new Employee("Anestis", "Tsalidis", -1, "Διευθυν", "Junior Developer", 0),
500             0.0, // expected AnnualGrossSalary
501             0.0, // expected NetAnnualIncome
502             0.0, // expected NetMonthIncome
503             0.0, // expected Tax
504             0.0, // expected Insurance
505         }
506     };
507     CalculateSalaryTestsCases(payroll, testcases);
508 }
509
510 [TestMethod]
511 // 0 references
512 public void TestCase_id_7()
513 {
514     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
515
516     // Collection of test cases for TestCase_id_2,3,4,5 with expected value 0 for all salary components
517     object[,] testcases =
518     {
519         // Case 6: Mid-level Developer, Children=4, WorkExp=15, Department=Υπόλληλος γραφείου (άκυρο)
520         {
521             7, // caseId
522             false, // expectedFlag for success
523             new Employee("Kostas", "Dedes", 0, "Διευθυν", "Manager", 0),
524             0.0, // expected AnnualGrossSalary
525             0.0, // expected NetAnnualIncome
526             0.0, // expected NetMonthIncome
527             0.0, // expected Tax
528             0.0, // expected Insurance
529         }
530     };
531     CalculateSalaryTestsCases(payroll, testcases);
532 }
533 }

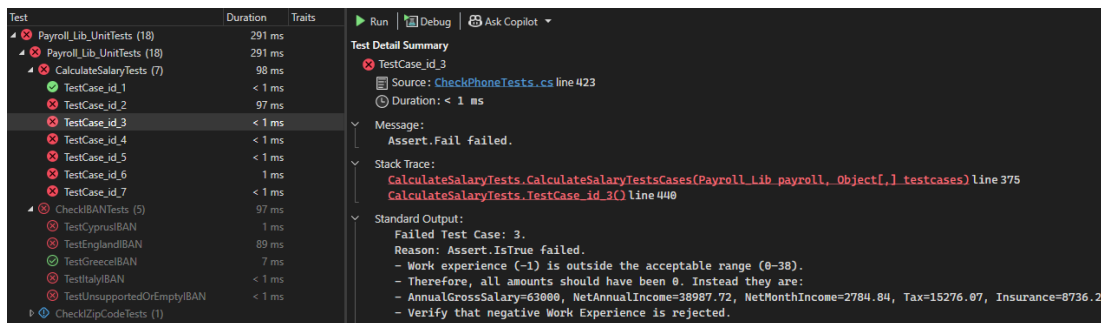
```

Δ. Test Reports

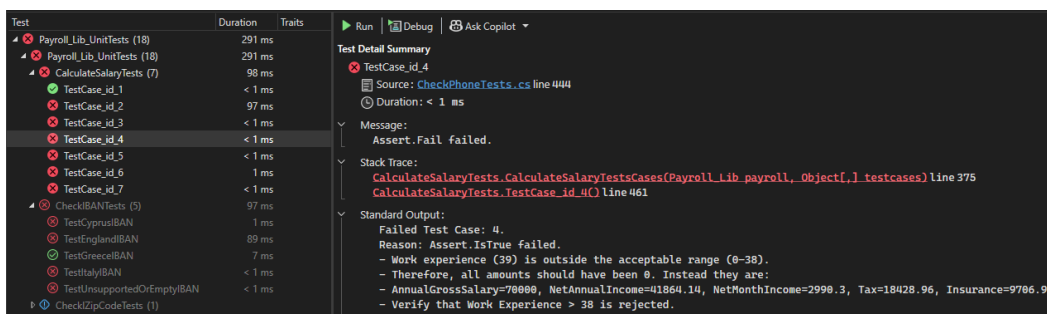
Test Explorer			Run Debug	
Test run finished: 7 Tests (1 Passed, 6 Failed, 0 Skipped) run in 440 ms			Test Detail Summary	
Test	Duration	Traits	TestCase_id_1	
Payroll_Lib_UnitTests (18)	290 ms		Source: CheckPhoneTests.cs line 381	
Payroll_Lib_UnitTests (18)	290 ms		Duration: < 1 ms	
CalculateSalaryTests (7)	97 ms			
Testcase_id_1	< 1 ms			
Testcase_id_2	96 ms			
Testcase_id_3	< 1 ms			
Testcase_id_4	< 1 ms			
Testcase_id_5	< 1 ms			
Testcase_id_6	1 ms			
Testcase_id_7	< 1 ms			



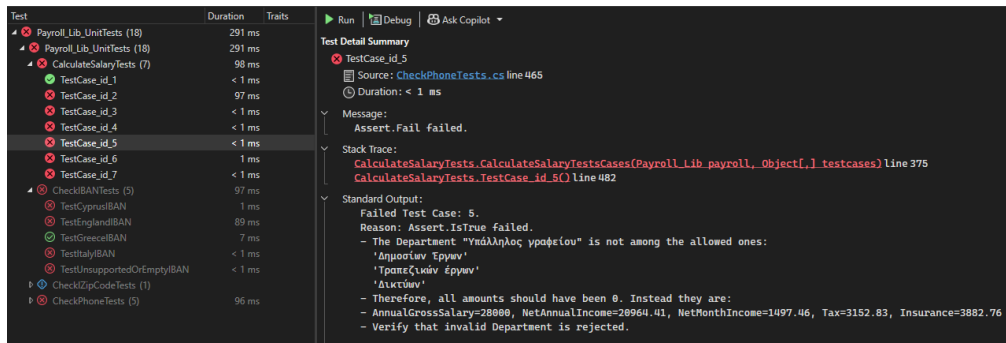
This is the test case with id:2. As shown in TestMethod = TestCase_id_2, it throws an error because the values that (AnnualGrossSalary, NetAnnualIncome, NetMonthIncome, Tax, Insurance) should have are not the accepted ones. Specifically, this occurred for the employee with Position: Senior Developer. Thus, the programmer must check if there is any logical error.



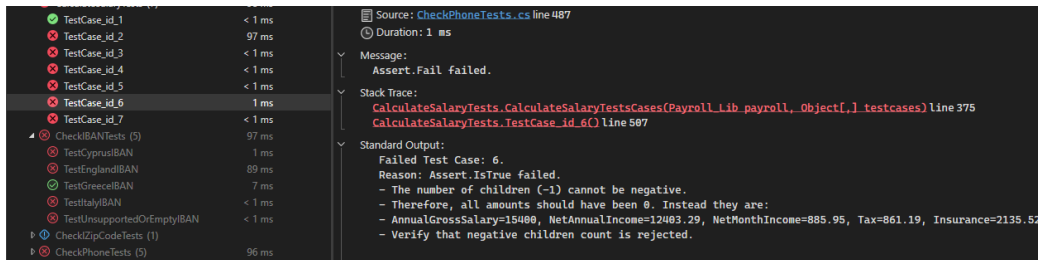
This is the test case with id:3. As shown in TestMethod = TestCase_id_3, it throws an error because the value of WorkExperience = -1 is not within the allowed bounds (0-38), and (AnnualGrossSalary, NetAnnualIncome, NetMonthIncome, Tax, Insurance) should have had the value of 0. Thus, the negative value of WorkExperience is not being checked.



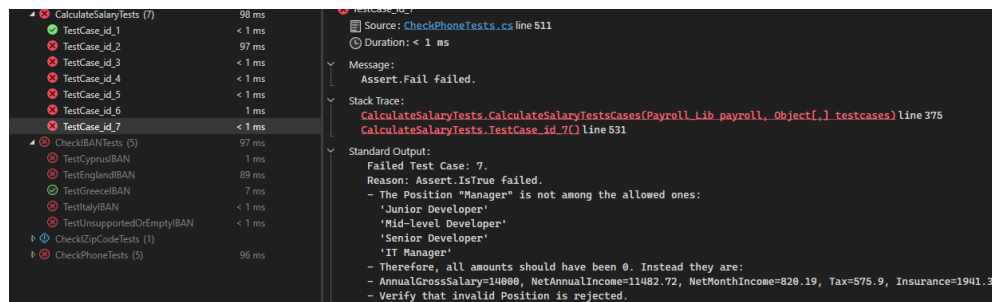
This is the test case with id:4. As shown in TestMethod = TestCase_id_4, it throws an error because the value of WorkExperience = 39 is not within the allowed bounds (0-38), and (AnnualGrossSalary, NetAnnualIncome, NetMonthIncome, Tax, Insurance) should have had the value of 0. Thus, the limit > 38 for WorkExperience is not being checked.



This is the test case with id:5. As shown in TestMethod = TestCase_id_5, it throws an error because the value of Department = “Office Clerk” is not among the allowed ones ('Public Works', 'Banking Projects', 'Networks'). The (AnnualGrossSalary, NetAnnualIncome, NetMonthIncome, Tax, Insurance) should have had the value of 0. Thus, a proper check is not taking place.



This is the test case with id:6. As shown in TestMethod = TestCase_id_6, it throws an error because the value of Children = -1 is a negative value and is not acceptable. The (AnnualGrossSalary, NetAnnualIncome, NetMonthIncome, Tax, Insurance) should have had the value of 0. Thus, the negative value of Children is not being checked.



This is the test case with id:7. As shown in TestMethod = TestCase_id_7, it throws an error because the value of Position = “Manager” is not among the allowed ones ('Junior Developer', 'Mid-level Developer', 'Senior Developer', 'IT Manager'). The (AnnualGrossSalary, NetAnnualIncome, NetMonthIncome, Tax, Insurance) should have had the value of 0. Thus, a proper check is not taking place.

5. int NumofEmployees (Employee [] Empls, string Position)

A. Method Implementation Assumptions

- This function returns the number of employees in a specific job position.
 - **Valid Job Positions (Position):** The method expects that the Position is one of the defined/allowed positions:
 - ❖ "Junior Developer"
 - ❖ "Mid-level Developer"
 - ❖ "Senior Developer"
 - ❖ "IT Manager"
- Any other value is considered invalid, and 0 must be returned as the output value of the method.
- The array Empls contains objects of type Employee. If the array is null, an Exception should be thrown. If it is empty (Empls.Length == 0), it should return 0.
 - If the Position is null or "", it usually returns 0.
 - The method counts objects in the employee array that possess the Position (Emp.Position == Position). If no employee with the specific position is found, it returns 0.

B. Test Case Creation

Equivalence Classes Table

Variable	Input Condition	Equivalence Classes	
		Valid	Invalid
Empls objects (employees)	of type Employee	• Non-empty array (at least 1 employee) • Empty array (0 employees)	null array
Position (job position)	of type string	"Junior Developer", "Mid-level Developer", "Senior Developer", "IT Manager"	• Null • Empty string • Any string other than the valid ones

Test Cases

Test Cases	Input Test Data		Expected Results
#id	Empls	Position	flag(int)

1	5 employees: 3 are "Junior Developer", 2 are "Senior Developer"	Junior Developer	3
2	6 employees: 2 are "Junior Developer", 4 are "IT Manager"	Senior Developer	0
3	0 employees	Junior Developer	0
4	3 employees: "Junior Developer"	Manager	0
5	null	Mid-level Developer	flag = 0 (expected error)
6	Employees	empty string	flag = 0 (expected error)
7	Employees	null	flag = 0 (expected error)

Г. Unit Test Development (UnitTests)

```

533 [TestClass]
534 0 references
535 public class NumOfEmployeesTests
536 {
537     7 references | 2/7 passing
538     private void NumOfEmployees_TestCases(Payroll_Lib.Payroll_Lib payroll, object[,] testcases)
539     {
540         bool failed = false;
541
542         for (int i = 0; i < testcases.GetLength(0); i++)
543         {
544             int id = (int)testcases[i, 0];
545             int expected = (int)testcases[i, 1];
546             Employee[] emps = (Employee[])testcases[i, 2];
547             string position = (string)testcases[i, 3];
548             string hint = (string)testcases[i, 4];
549
550             try
551             {
552                 Assert.AreEqual(expected, payroll.NumOfEmployees(emps, position));
553             }
554             catch (Exception ex)
555             {
556                 failed = true;
557                 Console.WriteLine("Failed Test Case: {0}. Position='{1}'.\nHint: {2}\nException: {3}",
558                     id, position ?? "null", hint, ex.Message);
559             }
560
561             if (failed) Assert.Fail();
562         }
563     }
564 }

```

```

363 [TestMethod]
364 public void TestCase1()
365 {
366     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
367     Employee[] Employee_testcase1 =
368     {
369         new Employee("Axil", "Rak", 0, "Διευθυν", "Junior Developer", 2),
370         new Employee("Basil", "Karast", 1, "Διευθυν", "Junior Developer", 2),
371         new Employee("Chrysa", "Chardal", 0, "Τραπεζικών έργων", "Junior Developer", 34),
372         new Employee("Dora", "Exer", 2, "Ανθοπωλιών Έργων", "Senior Developer", 4),
373         new Employee("Eirini", "Fivh", 0, "Διευθυν", "Senior Developer", 5),
374     };
375
376     object[,] testcases =
377     {
378         {1, 3, Employee_testcase1, "Junior Developer", "There are 3 Junior Developers employees in the Empls list. " +
379         "\nSo, it should have been a valid case. Make sure to verify your test conditions. " }
380     };
381
382     NumOfEmployees_TestCases(payroll, testcases);
383 }

```

```

585 [TestMethod]
586 public void TestCase2()
587 {
588     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
589     Employee[] Employee_testcase2 =
590     {
591         new Employee("Axil", "Rak", 0, "Διευθυν", "Junior Developer", 1),
592         new Employee("Basil", "Karast", 1, "Διευθυν", "Junior Developer", 2),
593         new Employee("Chrysa", "Chardal", 0, "Διευθυν", "IT Manager", 5),
594         new Employee("Dora", "Exer", 2, "Διευθυν", "IT Manager", 6),
595         new Employee("Eirini", "Fivh", 0, "Ανθοπωλιών Έργων", "IT Manager", 7),
596         new Employee("Xaris", "Oikon", 1, "Τραπεζικών έργων", "IT Manager", 4),
597     };
598
599     object[,] testcases =
600     {
601         {2, 0, Employee_testcase2, "Senior Developer", "In the specified case, there are no Senior Developer employees." +
602         "\nSo, it should have been a valid case. Make sure to verify your test conditions. " }
603     };
604
605     NumOfEmployees_TestCases(payroll, testcases);
606 }
607
608 [TestMethod]
609 public void TestCase3()
610 {
611     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
612
613     object[,] testcases =
614     {
615         {3, 0, new Employee[0], "Junior Developer", "The array of Employees is an empty string." +
616         "\nSo it should have returned 0. Make sure to verify your test conditions." }
617     };
618
619     NumOfEmployees_TestCases(payroll, testcases);
620 }
621

```

```

622 [TestMethod]
623 public void TestCase4()
624 {
625     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
626     Employee[] Employee_testcase4 =
627     {
628         new Employee("Axil", "Rak", 0, "Διευθυν", "Junior Developer", 1),
629         new Employee("Basil", "Karast", 1, "Διευθυν", "Junior Developer", 2),
630         new Employee("Chrysa", "Chardal", 0, "Διευθυν", "Junior Developer", 3),
631     };
632
633     object[,] testcases =
634     {
635         {4, 0, Employee_testcase4, "Manager", "The Position is non valid but it was passed as valid." +
636         "\nMake sure to verify your test conditions." }
637     };
638
639     NumOfEmployees_TestCases(payroll, testcases);
640 }
641
642 [TestMethod]
643 public void TestCase5()
644 {
645     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
646
647     object[,] testcases =
648     {
649         {5, 0, null, "Mid-level Developer", "The Employee array is null." +
650         "\nSo it should have returned 0. Make sure to verify your test conditions." }
651     };
652
653     NumOfEmployees_TestCases(payroll, testcases);
654 }

```

```

656 [TestMethod]
657 0 | 0 references
658 public void TestCase6()
659 {
660     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
661     Employee[] Employee_testcase1 =
662     {
663         new Employee("Axil", "Rak", 0, "Διευθυν", "Junior Developer", 2),
664         new Employee("Basil", "Karast", 1, "Διευθυν", "Junior Developer", 2),
665         new Employee("Chrysa", "Chardal", 0, "Τραπεζικών έργων", "Junior Developer", 34),
666         new Employee("Eirini", "Fivh", 2, "Ανθοπωλείων Έργων", "Senior Developer", 4),
667         new Employee("Xaris", "Oikon", 0, "Διευθυν", "Senior Developer", 5),
668     };
669
670     object[,] testcases =
671     {
672         {6, 0, Employee_testcase1, "", "The Position is an empty string. Make sure to verify your test conditions." }
673     };
674
675     NumOfEmployees_TestCases(payroll, testcases);
676 }
677
678 [TestMethod]
679 0 | 0 references
680 public void TestCase7()
681 {
682     Payroll_Lib.Payroll_Lib payroll = new Payroll_Lib.Payroll_Lib();
683     Employee[] Employee_testcase1 =
684     {
685         new Employee("Axil", "Rak", 0, "Διευθυν", "Junior Developer", 2),
686         new Employee("Basil", "Karast", 1, "Διευθυν", "Junior Developer", 2),
687         new Employee("Chrysa", "Chardal", 0, "Τραπεζικών έργων", "Junior Developer", 34),
688     };
689
690     object[,] testcases =
691     {
692         {7, 0, Employee_testcase1, null, "The Position is null. Make sure to verify your test conditions." }
693     };
694
695     NumOfEmployees_TestCases(payroll, testcases);
696 }

```

Δ. Test Reports

Test run finished: 7 Tests (2 Passed, 5 Failed, 0 Skipped) run in 429 ms		
Test	Duration	T
✗ Payroll_Lib_UnitTests (25)	480 ms	
✗ Payroll_Lib_UnitTests (25)	480 ms	
✗ CalculateSalaryTests (7)	95 ms	
✗ CheckIBANTests (5)	97 ms	
✗ CheckZipCodeTests (1)	96 ms	
✗ TestItalianZipCodes	96 ms	
✗ CheckPhoneTests (5)	96 ms	
✗ NumOfEmployeesTests (7)	96 ms	
✗ TestCase1	1 ms	
✗ TestCase2	< 1 ms	
✓ TestCase3	< 1 ms	
✗ TestCase4	< 1 ms	
✗ TestCase5	95 ms	
✗ TestCase6	< 1 ms	
✓ TestCase7	< 1 ms	

Run | Debug

Group Summary

NumOfEmployeesTests

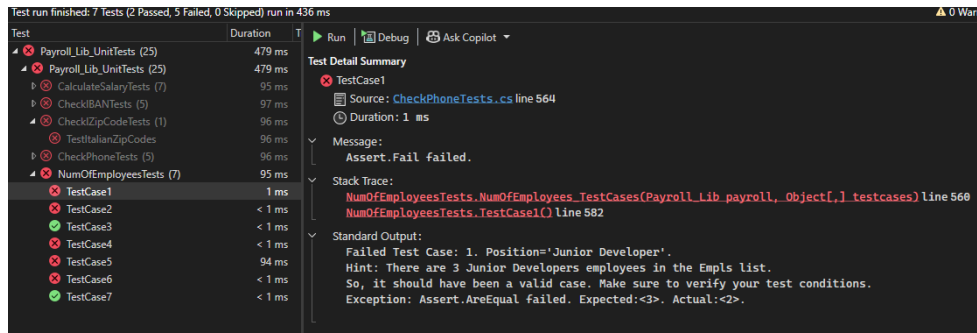
Tests in group: 7

⌚ Total Duration: 96 ms

Outcomes

✓ 2 Passed

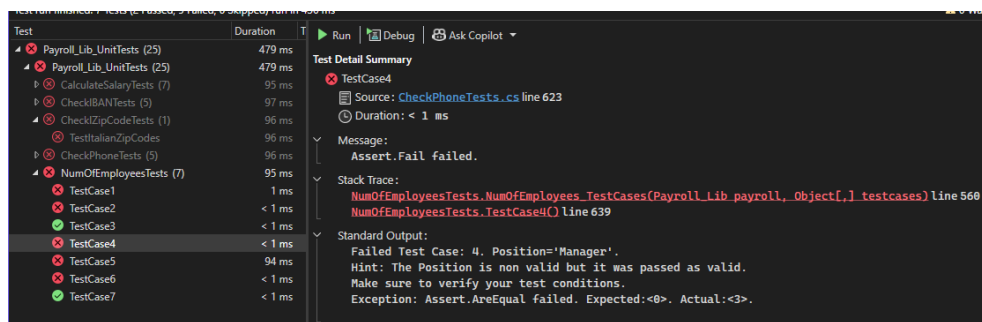
✗ 5 Failed



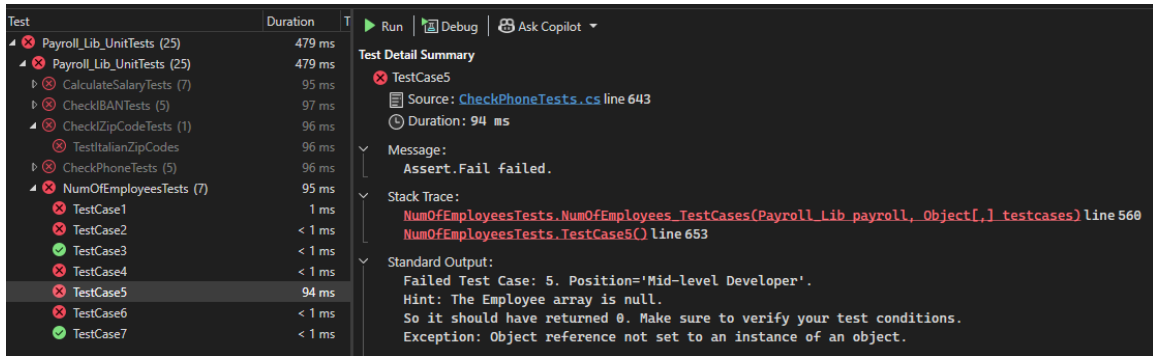
As shown in TestMethod = TestCase1, it throws an error for the number of employees who are “Junior Developers”. These employees are 3, but count=2 was returned. This means that the programmer made a mistake in the calculation. Therefore, it is a logical error.



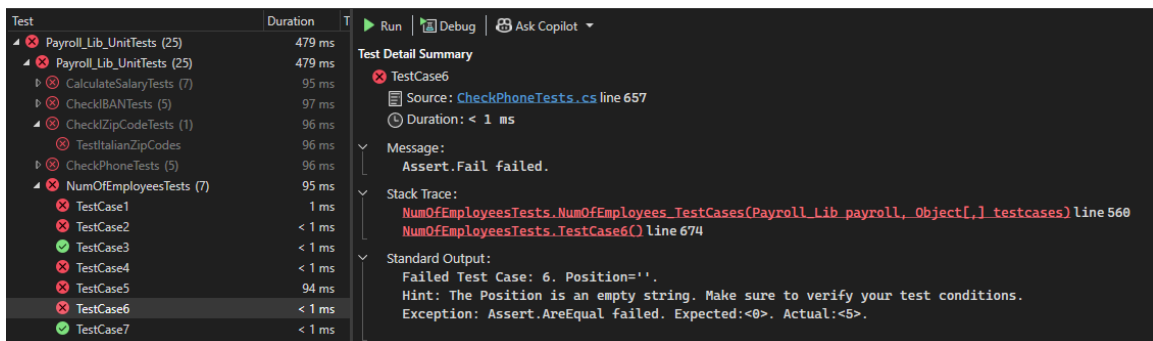
As shown in TestMethod = TestCase2, it throws an error for the number of employees who are “Senior Developers”. Normally, none of the 6 employees are Senior, so it should have returned count=0. This means that the programmer made a mistake in the calculation. Therefore, it is a logical error.



As shown in TestMethod = TestCase4, it throws an error for the number of employees. The Manager Position does not exist, so count=0 should have been returned. This means that the programmer is not checking the job positions. Therefore, it is a logical error.



As shown in TestMethod = TestCase5, it throws an error regarding the object reference. The Employees array is null. Thus, count=0 should have been returned. This means that the programmer is not checking for the null array case. Therefore, it is a logical error.



As shown in TestMethod = TestCase6, it throws an error for the number of employees. The position is an empty string. Thus, count=0 should have been returned. This means that the programmer is not checking for the empty string case. Therefore, it is a logical error.

6. bool GetBonus(ref Employee[] Empls, string Department, double IncomeGoal, double Bonus)

A. Method Implementation Assumptions

- The method accepts an array of employees (Employee[]), the company department (string Department), the revenue target (double IncomeGoal), and the department's bonus (double Bonus).
- Initially, the method performs validation checks on the input parameters::
 - ❖ It returns false if the employee array is null or empty.
 - ❖ It returns false if the department is empty (null or string.Empty).
 - ❖ It returns false if the revenue target or the total bonus is zero or a negative value.
- The method calculates the total revenue for the specific department by summing the Income field value for each employee.
- If no employee is found for the specific department, the method returns false.
- If the total revenue exceeds the target (IncomeGoal), then the method:
 - ❖ Calculates each employee's contribution percentage by dividing the employee's income (Income) by the department's total revenue.
 - ❖ Calculates each employee's Bonus by multiplying the contribution percentage by the bonus.
 - ❖ Updates the Bonus field of each employee within the array.
 - ❖ Returns true.
- If the department's total revenue is less than the target (IncomeGoal), then:
 - ❖ Each employee's bonus becomes 0.
 - ❖ The method returns false.

B. Test Case Creation

Equivalence Classes Table

Variable	Input Condition	Equivalence Classes	
		Valid	Invalid
Empls objects (employees)	of type Employee	- Non-empty array (at least 1 employee), - Empty array (0 employees)	null array
Department	of type string	"Public Works", "Banking Projects", "Networks"	- Null, - Empty string, - Any string other than the valid ones
IncomeGoal (revenue target)	of type double	Positive number	- Null, - Empty string, - Any string other than the valid ones
Department Bonus	of type double	Positive number	- Negative number, 0

Test Cases

Test Cases	Input Test Data				Expected Results	
#id	Empls	Department	IncomeGoal	Bonus	flag(bool)	Empls.Bonus
1	3 employees with total revenue: 14000	"Networks"	12000	1400	true	The calculated Bonus
2	6 employees with total revenue: 15000	"Banking Projects"	17000	1200	false	0
3	0 employees	"Banking Projects"	10000	1100	false	Remains 0
4	null employee array	"Public Works"	11000	1000	false	Remains 0
5	4 employees with total revenue: 10000	empty string	10000	1000	false	Remains 0
6	5 employees with total revenue: 8000	"Networks"	0	1000	false	Remains 0
7	3 employees with total revenue: 23000	"Networks"	22000	0	false	Remains 0

F. Unit Test Development (UnitTests)

```
697 [TestClass]
698 0 references
699 public class GetBonusTests
700 {
701     7 references | 2/7 passing
702     private void GetBonusTestCases(Payroll_Lib.Payroll_Lib payroll, object[,] testcases)
703     {
704         bool failed = false;
705
706         for (int i = 0; i < testcases.GetLength(0); i++)
707         {
708             int caseId = (int)testcases[i, 0];
709             bool expectedFlag = (bool)testcases[i, 1];
710             Employee[] Empls = (Employee[])testcases[i, 2];
711             string Department = (string)testcases[i, 3];
712             double IncomeGoal = (double)testcases[i, 4];
713             double Bonus = (double)testcases[i, 5];
714             double[] expectedBonuses = (double[])testcases[i, 6];
715             string hint = (string)testcases[i, 7];
716
717             try
718             {
719                 Employee[] Empls_copy = null;
720                 if (Empls != null)
721                 {
722                     Empls_copy = new Employee[Empls.Length];
723                     for (int j = 0; j < Empls.Length; j++)
724                         Empls_copy[j] = Empls[j];
725
726                     bool actualFlag = payroll.GetBonus(ref Empls_copy, Department, IncomeGoal, Bonus);
727
728                     string errorMsg = "";
729
730                     // Flag check
731                     if (actualFlag != expectedFlag)
732                         errorMsg += $"Expected:<{expectedFlag}>. Actual:<{actualFlag}>.\n";
733
734                     // Empty department check
735                     if (Empls_copy != null)
736                     {
737                         for (int j = 0; j < Empls_copy.Length; j++)
738                         {
739                             if (Empls_copy[j].Department == "")
740                                 errorMsg += $"- Employee {j+1} doesn't have a Department.\n";
741                         }
742
743                         // Bonus check
744                         if (Empls_copy != null && expectedBonuses != null)
745                         {
746                             string bonusErrors = "";
747                             for (int j = 0; j < Empls_copy.Length; j++)
748                             {
749                                 double expected = expectedBonuses[j];
750                                 double actual = Empls_copy[j].Bonus;
751                                 if (Math.Abs(actual - expected) > 0.01)
752                                     bonusErrors +=
753                                         $"- Emp[{j}] {Empls_copy[j].FirstName} {Empls_copy[j].Surname} "
754                                         + $"expected {expected} got {actual}\n";
755                             }
756
757                             if (!string.IsNullOrEmpty(bonusErrors))
758                                 errorMsg += $"Bonus mismatches:\n{bonusErrors}";
759                         }
760
761                         if (!string.IsNullOrEmpty(errorMsg))
762                             Assert.Fail($"errorMsg");
763                     }
764                     catch (Exception ex)
765                     {
766                         failed = true;
767                         Console.WriteLine(
768                             "Failed Test Case: {0}. Hint: {1}\nException: {2}",
769                             caseId, hint, ex.Message);
770                     }
771                 }
772                 if (failed) Assert.Fail();
773             }
774         }
775     }
776 }
```

```

775 [TestMethod]
776 public void GetBonus_TestCase1()
777 {
778     var payroll = new Payroll_Lib.Payroll_Lib();
779     Employee[] Empls =
780     {
781         new Employee("Christos","Skiadaresis",0,"Δικτύων","Junior Developer",3,4000,0),
782         new Employee("Katerina","Giorgatzi",0,"Δικτύων","Mid-level Developer",4,5000,0),
783         new Employee("Giorgos","Cheimonidis",0,"Δικτύων","Senior Developer",6,5000,0)
784     };
785     double[] expectedBonuses = { 400.0, 500.0, 500.0 };
786     /*
787         Employee 1: (4000 / 14000) * 1400 = 0.285714 * 1400 = 400.00
788         Employee 2: (5000 / 14000) * 1400 = 0.357142 * 1400 = 500.00
789         Employee 3: (5000 / 14000) * 1400 = 0.357142 * 1400 = 500.00
790     */
791     object[,] testcases =
792     {
793         {1, true, Empls, "Δικτύων", 12000.0, 1400.0, expectedBonuses,"It should have returned true."}
794     };
795     GetBonusTestCases(payroll, testcases);
796 }
797
798 [TestMethod]
799 public void GetBonus_TestCase2()
800 {
801     var payroll = new Payroll_Lib.Payroll_Lib();
802     Employee[] Empls =
803     {
804         new Employee("Christos","Skiadaresis",0,"Τραπεζικών έργων","Junior Developer",2,3000,100),
805         new Employee("Katerina","Giorgatzi",0,"Τραπεζικών έργων","Mid-level Developer",4,2000,200),
806         new Employee("Giorgos","Cheimonidis",0,"Τραπεζικών έργων","Senior Developer",6,2500,150),
807         new Employee("Chrysa","Chardaloupa",0,"Τραπεζικών έργων","Junior Developer",1,1500,120),
808         new Employee("Dora","Exen",0,"Τραπεζικών έργων","Mid-level Developer",3,3500,130),
809         new Employee("Nikos","Kalofwnos",0,"Τραπεζικών έργων","Senior Developer",5,2500,110)
810     };
811     double[] expectedBonuses = { 0, 0, 0, 0, 0, 0 };
812     object[,] testcases =
813     {
814         {2, false, Empls, "Τραπεζικών έργων", 17000.0, 1200.0, expectedBonuses,"It should have been false because " +
815             "IncomeGoal > than total income from all Employees."}
816     };
817     GetBonusTestCases(payroll, testcases);
818 }

```

```

820 [TestMethod]
821 public void GetBonus_TestCase3()
822 {
823     var payroll = new Payroll_Lib.Payroll_Lib();
824     Employee[] Empls = new Employee[0];
825     double[] expectedBonuses = new double[0];
826     object[,] testcases =
827     {
828         {3, false, Empls, "Τραπεζικών έργων", 10000.0, 1100.0, expectedBonuses,"It should have returned 'false' because\n" +
829             "the array of employees (Empls) is empty."}
830     };
831     GetBonusTestCases(payroll, testcases);
832 }
833
834 [TestMethod]
835 public void GetBonus_TestCase4()
836 {
837     var payroll = new Payroll_Lib.Payroll_Lib();
838     object[,] testcases =
839     {
840         {4, false, null, "Δημοσίων Έργων", 11000.0, 1000.0, null,"It should have returned 'false' because\n" +
841             "the array of employees (Empls) is null."}
842     };
843     GetBonusTestCases(payroll, testcases);
844 }

```

```

846 [TestMethod]
847 • | 0 references
848 public void GetBonus_TestCase5()
849 {
850     var payroll = new Payroll_Lib.Payroll_Lib();
851     Employee[] Empls =
852     {
853         new Employee("Christos","Skiadarexis",0,"", "Junior Developer",1,2000,0),
854         new Employee("Katerina","Giorgatzi",0,"", "Mid-level Developer",2,3000,0),
855         new Employee("Giorgos","Cheimonidis",0,"", "Senior Developer",3,2500,0),
856         new Employee("Nikos","Kalofwnos",0,"", "Junior Developer",1,2500,0)
857     };
858     double[] expectedBonuses = { 0, 0, 0, 0 };
859     object[,] testcases =
860     {
861         {5, false, Empls, "", 10000.0, 1000.0, expectedBonuses, "It should have returned 'false' because\n" +
862             "the employees exist but the department doesn't."}
863     };
864     GetBonusTestCases(payroll, testcases);
865 }
866
867 [TestMethod]
868 • | 0 references
869 public void GetBonus_TestCase6()
870 {
871     var payroll = new Payroll_Lib.Payroll_Lib();
872     Employee[] emps =
873     {
874         new Employee("Christos","Skiadarexis",0,"Διεύθυν", "Junior Developer",1,1000,0),
875         new Employee("Katerina","Giorgatzi",0,"Διεύθυν", "Junior Developer",1,1500,0),
876         new Employee("Giorgos","Cheimonidis",0,"Διεύθυν", "Mid-level Developer",3,2000,0),
877         new Employee("Nikos","Kalofwnos",0,"Διεύθυν", "Senior Developer",5,1500,0),
878         new Employee("Dora","Exer",0,"Διεύθυν", "Junior Developer",2,2000,0)
879     };
880     double[] expectedBonuses = { 0, 0, 0, 0, 0 };
881     object[,] testcases =
882     {
883         {6, false, emps, "Διεύθυν", 0.0, 1000.0, expectedBonuses, "It should have returned 'false' because the IncomeGoal is 0."}
884     };
885     GetBonusTestCases(payroll, testcases);
886 }

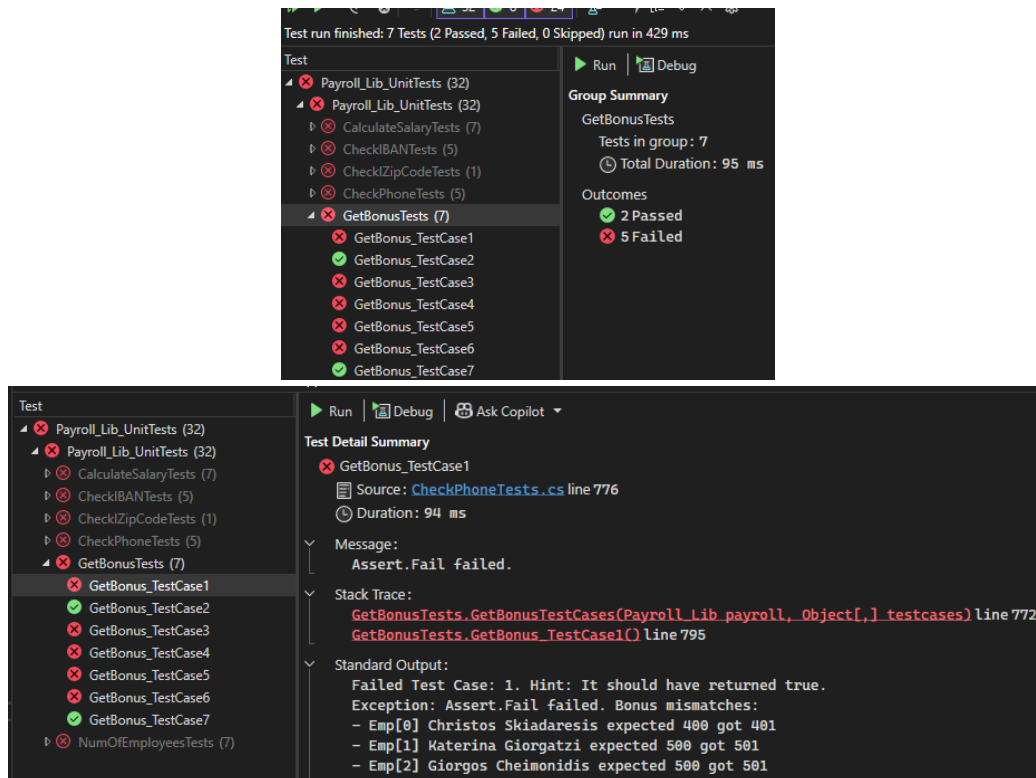
```

```

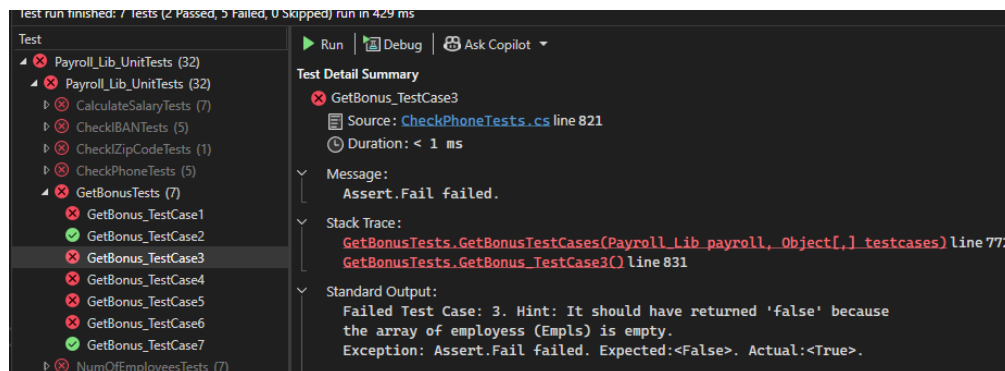
885 [TestMethod]
886 • | 0 references
887 public void GetBonus_TestCase7()
888 {
889     var payroll = new Payroll_Lib.Payroll_Lib();
890     Employee[] emps =
891     {
892         new Employee("Christos","Skiadarexis",0,"Διεύθυν", "Junior Developer",2,8000,0),
893         new Employee("Katerina","Giorgatzi",0,"Διεύθυν", "Mid-level Developer",4,7000,0),
894         new Employee("Giorgos","Cheimonidis",0,"Διεύθυν", "Senior Developer",6,8000,0)
895     };
896     double[] expectedBonuses = { 0, 0, 0 };
897     object[,] testcases =
898     {
899         {7, false, emps, "Διεύθυν", 22000.0, 0.0, expectedBonuses,
900             "Αποτυχία: bonus pool = 0."}
901     };
902     GetBonusTestCases(payroll, testcases);
903 }
904 }

```

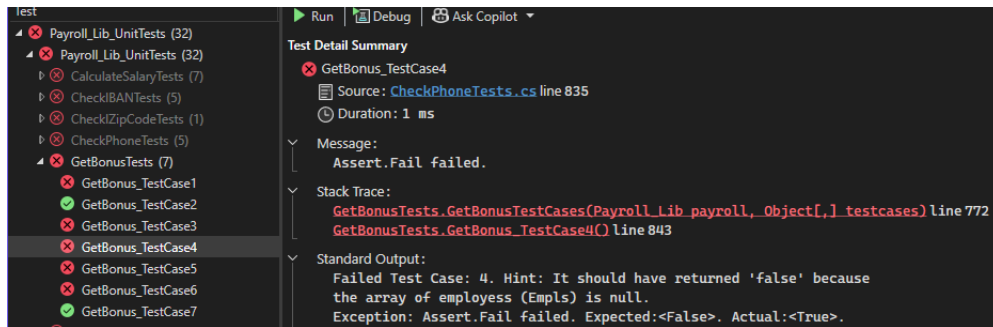
Δ. Test Reports



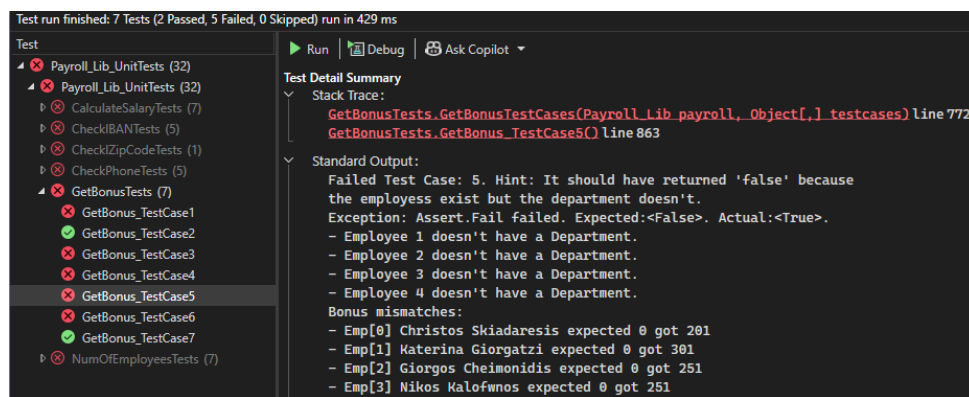
As shown in TestMethod = GetBonus_TestCase1, it should have returned true. This does not happen because the bonuses are calculated incorrectly. Therefore, it is a logical error and the calculation of each employee's bonus must be checked by the programmer.



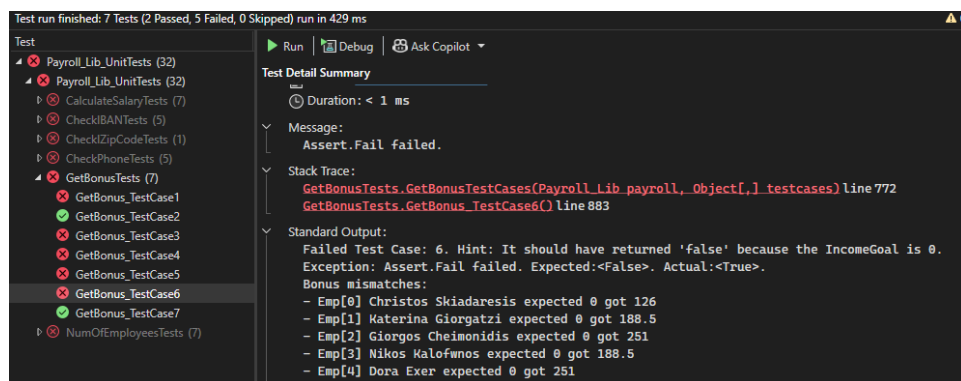
As shown in TestMethod = GetBonus_TestCase3, it should have returned false because the Employees array is empty. This is due to a logical error and must be checked by the programmer.



As shown in TestMethod = GetBonus_TestCase4, it should have returned false because the Employees array is null. This is due to a logical error and must be checked by the programmer.



As shown in TestMethod = GetBonus_TestCase5, it should have returned false because the Employees array has no Department for any Employee. Calculations are taking place, while the bonuses should have remained 0. These are due to logical errors and must be checked by the programmer.



As shown in TestMethod = GetBonus_TestCase6, it should have returned false because the IncomeGoal is 0. There should be a certain IncomeGoal amount, otherwise, there would be no reason for the GetBonus method to exist.